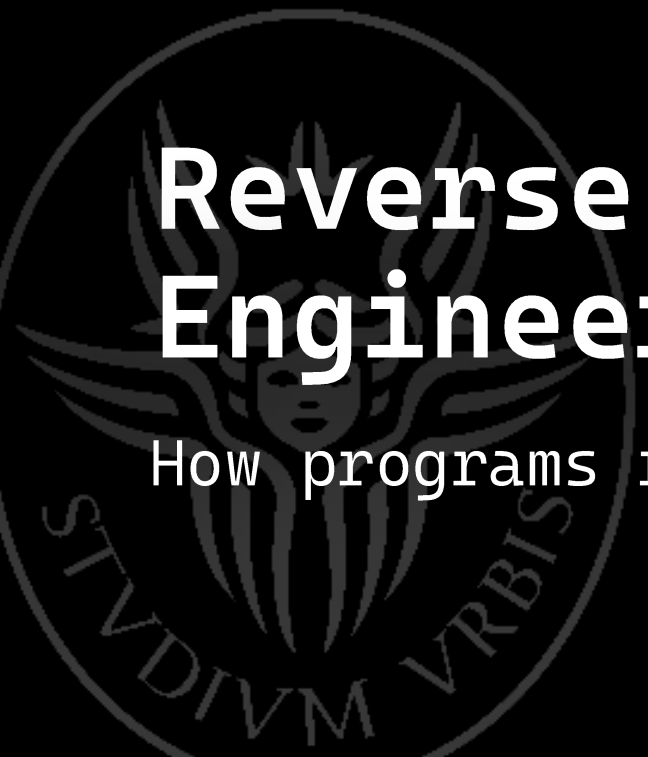
CYBER CHALLENGE.IT



Reverse Engineering

How programs run



About Us – Binary Version



About Us

About Us - (AI Generated) Hacker Version



Ice Cream



Kristoj



MagicFrank



Titto



Lorenzinco



Loldemort

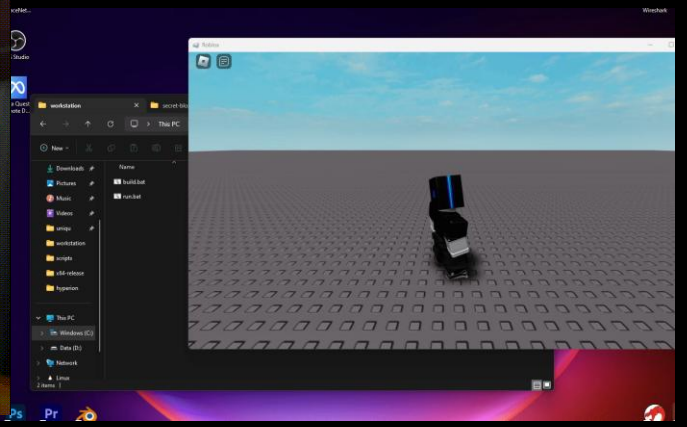
SPECIAL GUEST

CyberChallenge.IT - Sapienza Università di Roma

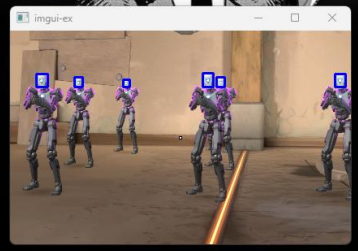
Rev – The Fun Side



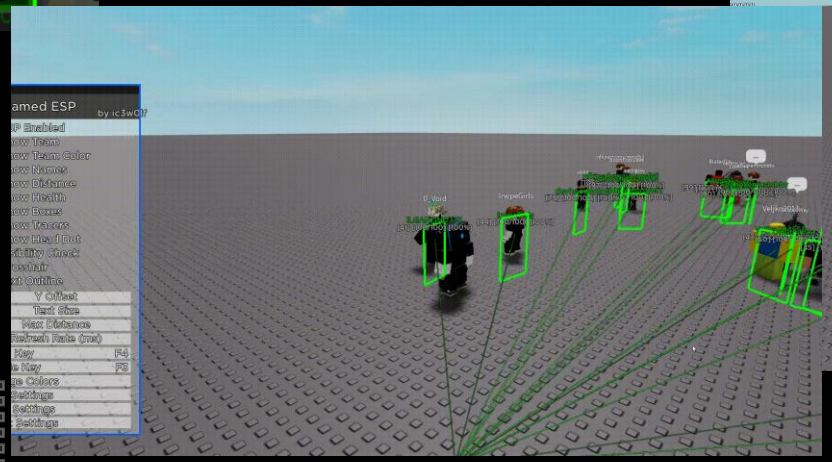




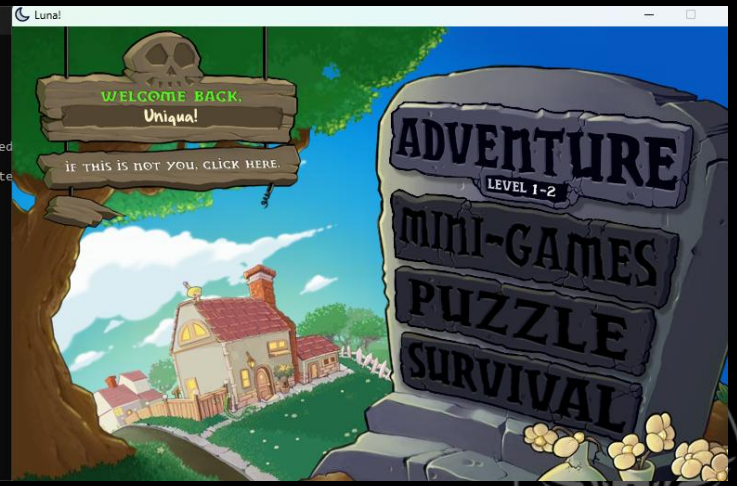
















Executable and Linkable Format

0b00101110 0b01000101 0b01001100 0b01000110

0x7f 0x45 0x4c 0x46

.ELF

These are called
Magic Numbers



LET'S READ IT

Usually an ELF looks like this ...



0xFile — lorenzocolombini@MacBook-Pro — ..oiting/0xFile — -zsh — 109×30

```
~/documents/progetti/CyberChallenge/exploiting/0xFile ls
0xfile     0xfile.i64 x.py
~/documents/progetti/CyberChallenge/exploiting/0xFile cat 0xfile
ELF>`@@?@8      @@@@@@?88@8@@@?
                              ?
                                ``h? ``?TT@T@DDP?tdl
l
@l
@DDQ?tdR?td``/lib64/ld-linux-x86-64.so.2GNUGNUнn?-??tr?lm{??)
?? P
(?BE???|?qX??????TCr&3w? k|??h`??`?h`[
                                       H@ 4      @libseccomp.so.2__gmon_start___finiseccomp_loadseccomp_releas
eseccomp_rule_addseccomp_initlibc.so.6fflushputsreadstdout__libc_start_main_edata__bss_start_endGLIBC_2.2.5au
i     ??`??`
` `(`0`8`@`H`   H?H??    H??t??H???5?    ?%?     @?%?    h??????%?       h??????%?  h??????%?       h?????%?
       h?????%z       h?????%r       h????1?I??^H??H???P]?h`??D]?fD?h`UH??h`H??H??H??H???H?H??t?H??t
                                      ]?h`??]?fD?= uUH???~???? ]?D??@f.?UH??]?UH??H?? ?E?;?E?B??????H?E
?H?E??(H?E??T??H?E??H?Ǹh???H?E?H?}?v?H?E?H???p???H?E?H???t??????fDAWAVI??AUATL?%. UH?-. SA??I??L)?H?H???W???
H??t 1??L??L??D??A??H??H9?u?H?[]A\A]A^A_Ðf.???H?H??Store your secret in our super secure enclave (100x more s
ecure than SGX):Good! Now that your data are in a secure storage please tell me an indetifier for it:seccomp_
blacklist() uses a linux kernel security feature in order to block execve and execveat syscalls for this proc
ess@?????t???\?????[?????????T???????LzRx
                                                                    ???+zRx
   ???$0P????FJ
```





ASM >>> Machine Code

Instructions are bytes
that correspond to ASM.

0xFile — lorenzocolombini@MacBook-Pro — ..oiting/0xFile — -zsh — 109×30

```
~/documents/progetti/CyberChallenge/exploiting/0xFile objdump -d 0xfile

0xfile:     file format elf64-x86-64


Disassembly of section .init:

0000000000400648 <_init>:
  400648:       48 83 ec 08             sub    $0x8,%rsp
  40064c:       48 8b 05 a5 09 20 00    mov    0x2009a5(%rip),%rax        # 600ff8 <__gmon_start__>
  400653:       48 85 c0                test   %rax,%rax
  400656:       74 02                   je     40065a <_init+0x12>
  400658:       ff d0                   call   *%rax
  40065a:       48 83 c4 08             add    $0x8,%rsp
  40065e:       c3                      ret

Disassembly of section .plt:

0000000000400660 <.plt>:
  400660:       ff 35 a2 09 20 00       push   0x2009a2(%rip)        # 601008 <_GLOBAL_OFFSET_TABLE_+0x8>
  400666:       ff 25 a4 09 20 00       jmp    *0x2009a4(%rip)        # 601010 <_GLOBAL_OFFSET_TABLE_+0x10>
```





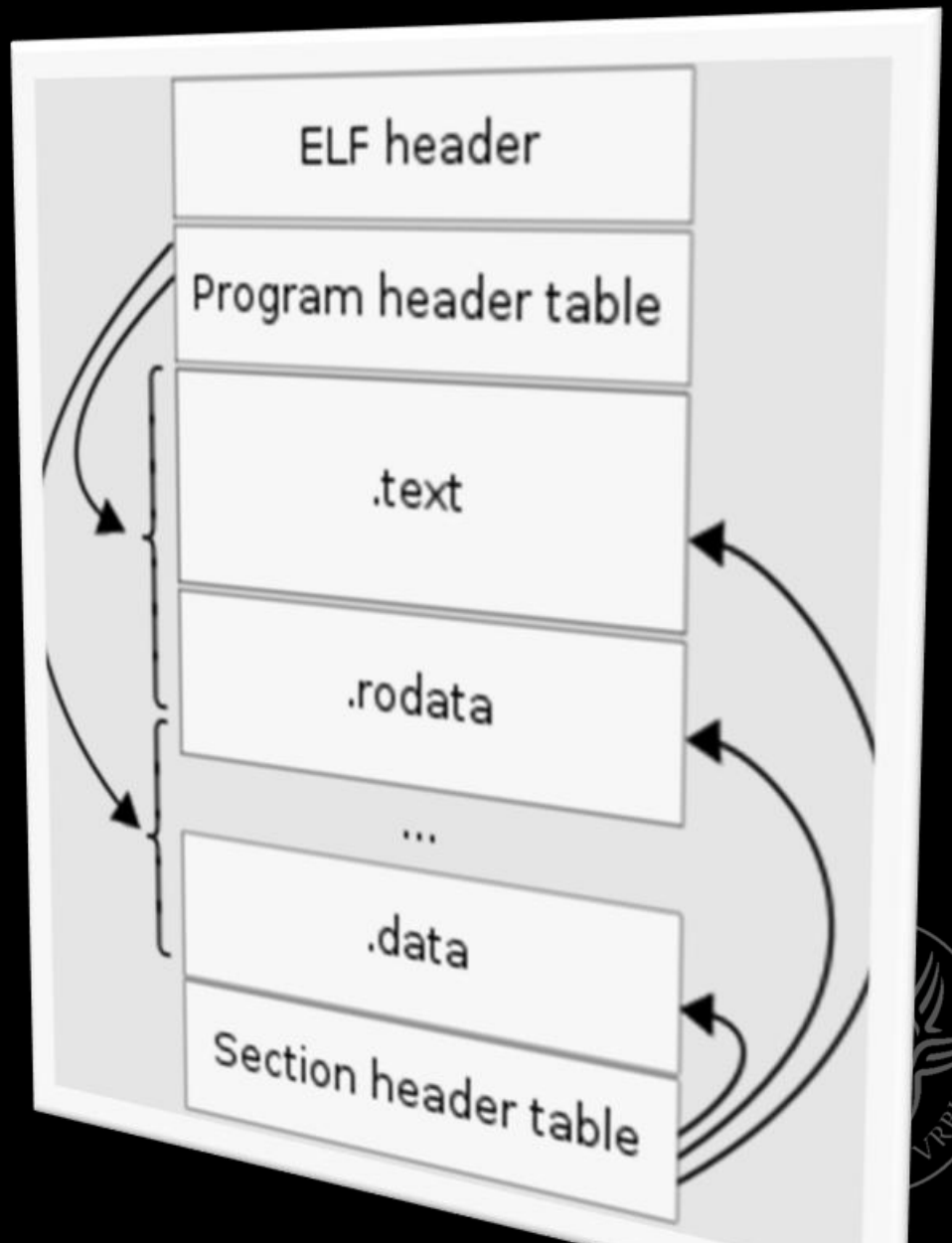
What's in a binary?

PROGRAM HEADER:
Shows the segments used

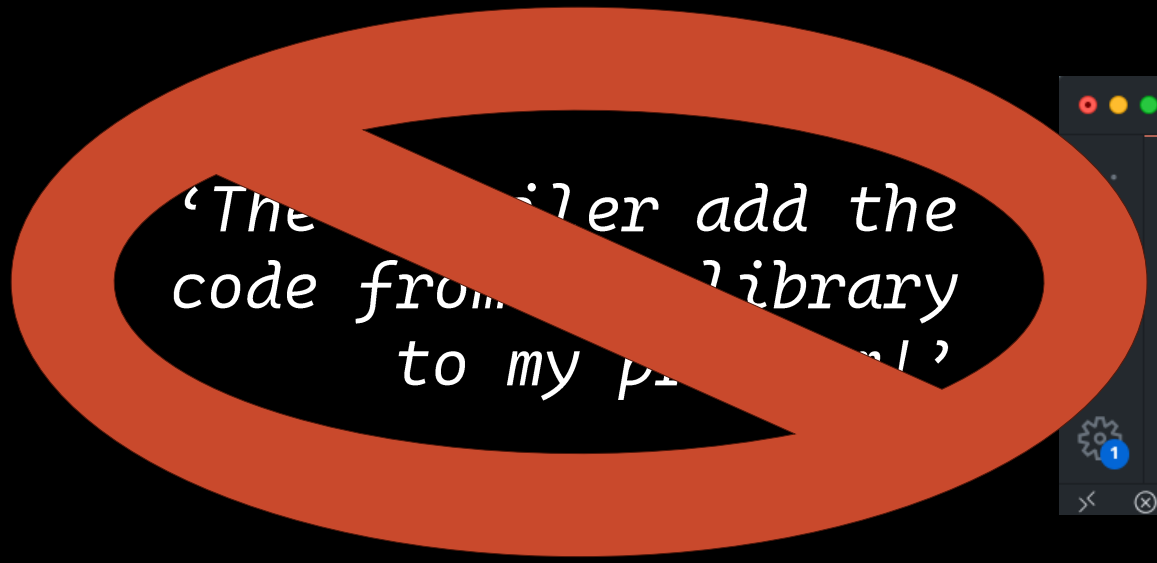
SEGMENTS:
Information for runtime-exec
Can keep sections

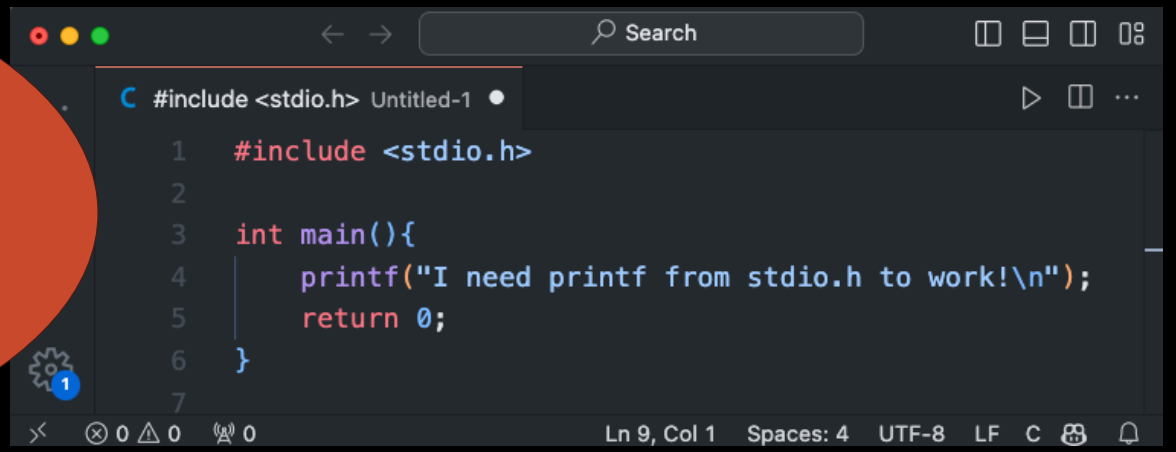
SECTIONS:
Data for linking and relocation

SECTION HEADER:
Lists the set of sections



Dynamic Linking






```
KRISTOJ:~/trx/workdir$ nm hello_world.o
0000000000000000 T main
                 U printf
```

```
KRISTOJ:~/trx/workdir$ nm hello_world
0000000000001149 T main
                 U printf@GLIBC_2.2.5
00000000000010c0 t register_tm_clones
```

GOT & PLT

Global Offset Table: external func addresses
Procedure Linkage Table: GOT addresses

Each external func call is compiled as a call to the PLT that works like this:

```
[ DISASM / x86-64 / set emulate on ]
lea     rax, [rip + 0xeac]          <main>
mov     rdi, rax
mov     eax, 0
call    printf@plt                  <printf@plt>
```

```
if GOT.func != empty :
    jump to lib[GOT.func.addr]

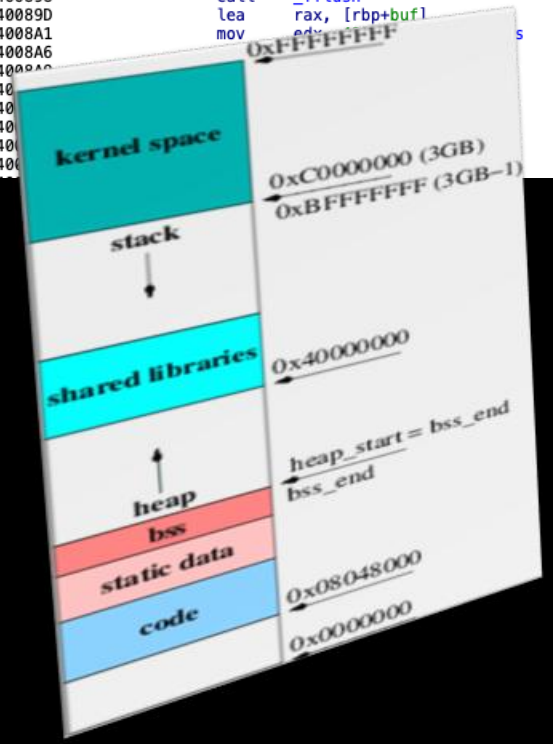
else
    coordinate with ld.so (dynamic linker)
    to get the func address
    GOT.func.addr <- address
    jump to lib[GOT.func.addr]
```





GOT & PLT

```
t:0000000000400864                mov     rdi, rax        ; stream
t:0000000000400867                call    _fflush
t:000000000040086C                mov     edx, 400h       ; nbytes
t:0000000000400871                lea     rsi, enclave    ; buf
t:0000000000400878                mov     edi, 0          ; fd
t:000000000040087D                call    _read
t:0000000000400882                lea     rdi, aGoodNowThatYou ; "Good! Now
t:0000000000400889                call    _puts
t:000000000040088E                mov     rax, cs:stdout@@GLIBC_2_2_5
t:0000000000400895                mov     rdi, rax        ; stream
t:0000000000400898                call    _fflush
t:000000000040089D                lea     rax, [rbp+buf]
t:00000000004008A1                mov     edx, 
t:00000000004008A6
```




```
.plt:0000000000400690
.plt:0000000000400690 ; int puts(const char *s)
.plt:0000000000400690 _puts           proc near               ; CODE XREF: main+19↓p
.plt:0000000000400690                                         ; main+4A↓p
.plt:0000000000400690                 jmp     cs:off_601028
.plt:0000000000400690 _puts           ends
```

```
.got.plt:0000000000601000 ; ===========================================================================
.got.plt:0000000000601000
.got.plt:0000000000601000 ; Segment type: Pure data
.got.plt:0000000000601000 ; Segment permissions: Read/Write
.got.plt:0000000000601000 _got_plt        segment qword public 'DATA' use64
.got.plt:0000000000601000                 assume cs:_got_plt
.got.plt:0000000000601000                 ;org 601000h
.got.plt:0000000000601000 _GLOBAL_OFFSET_TABLE_ dq offset _DYNAMIC
.got.plt:0000000000601008 qword_601008    dq 0                  ; DATA XREF: sub_400660↑r
.got.plt:0000000000601010 ; __int64 (*qword_601010)(void)
.got.plt:0000000000601010 qword_601010    dq 0                  ; DATA XREF: sub_400660+6↑r
.got.plt:0000000000601018 off_601018      dq offset seccomp_init  ; DATA XREF: _seccomp_init↑r
.got.plt:0000000000601020 off_601020      dq offset seccomp_rule_add
.got.plt:0000000000601020                                       ; DATA XREF: _seccomp_rule_add↑r
.got.plt:0000000000601028 off_601028      dq offset puts        ; DATA XREF: _puts↑r
.got.plt:0000000000601030                 dq offset seccomp_load  ; DATA XREF: _seccomp_load↑r
.got.plt:0000000000601038 off_601038      dq offset seccomp_release
.got.plt:0000000000601038                                       ; DATA XREF: _seccomp_release↑r
.got.plt:0000000000601040 off_601040      dq offset read        ; DATA XREF: _read↑r
.got.plt:0000000000601048 off_601048      dq offset fflush      ; DATA XREF: _fflush↑r
.got.plt:0000000000601048 _got_plt        ends
.got.plt:0000000000601048
.data:0000000000601050 ; ===========================================================================
```

Makes sense right?



If not, check out this: https://www.cprogramming.com/tutorial/shared-libraries-linux-gcc.html



FILE / STRINGS / BINWALK

Useful command line tools:

- $: strings elf | grep –i flag

```
|KRISTOJ|:~/trx/preCC/chall/rev/where_banana/sol$ strings wh3r3_b4n4n4
/lib64/ld-linux-x86-64.so.2
#tL+
mgUa
__cxa_finalize
__libc_start_main
stdout
puts
```

- $: binwalk --dd='.*’ elf



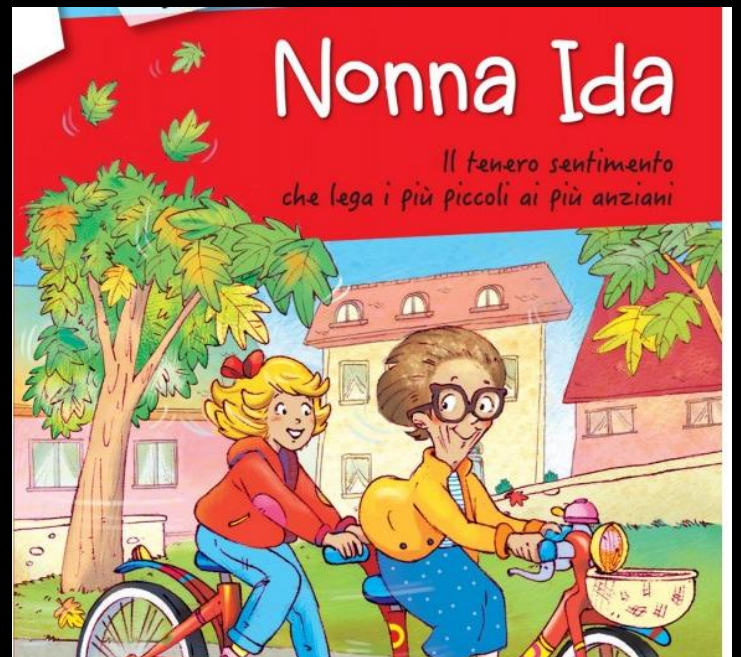
```
DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0x0             ELF, 64-bit LSB shared object, AMD x86-64, version 1
12423         0x3087          JPEG image data, JFIF standard 1.01
4528640       0x451A00        GIF image data, version "89a", 500 x 500
```



That’s already better but ...

- ASM still isn’t very user friendly ...
- Complex functions can take hours to reverse
- Someone already solved this problem:

IDA



That’s already better but ...

- ASM still isn’t very user friendly ...
- Complex functions can take hours to reverse
- Someone already solved this problem:

IDA



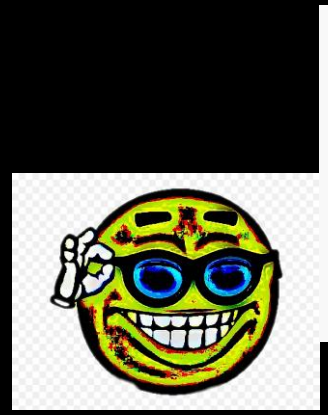


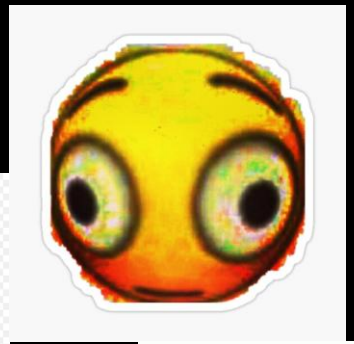


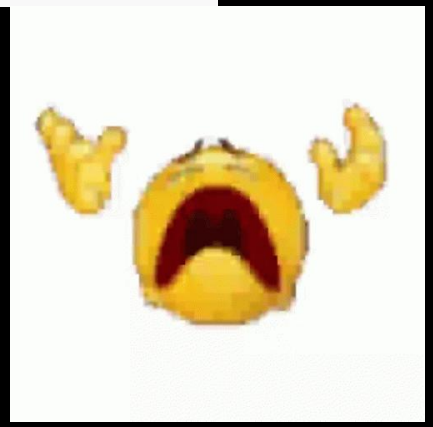
Introducing IDA



- Decompiler with simple (meh) UI
- Can generate C-equivalent from ASM
- Costs 40.000$
- Free version







- Does not cook on Sundays



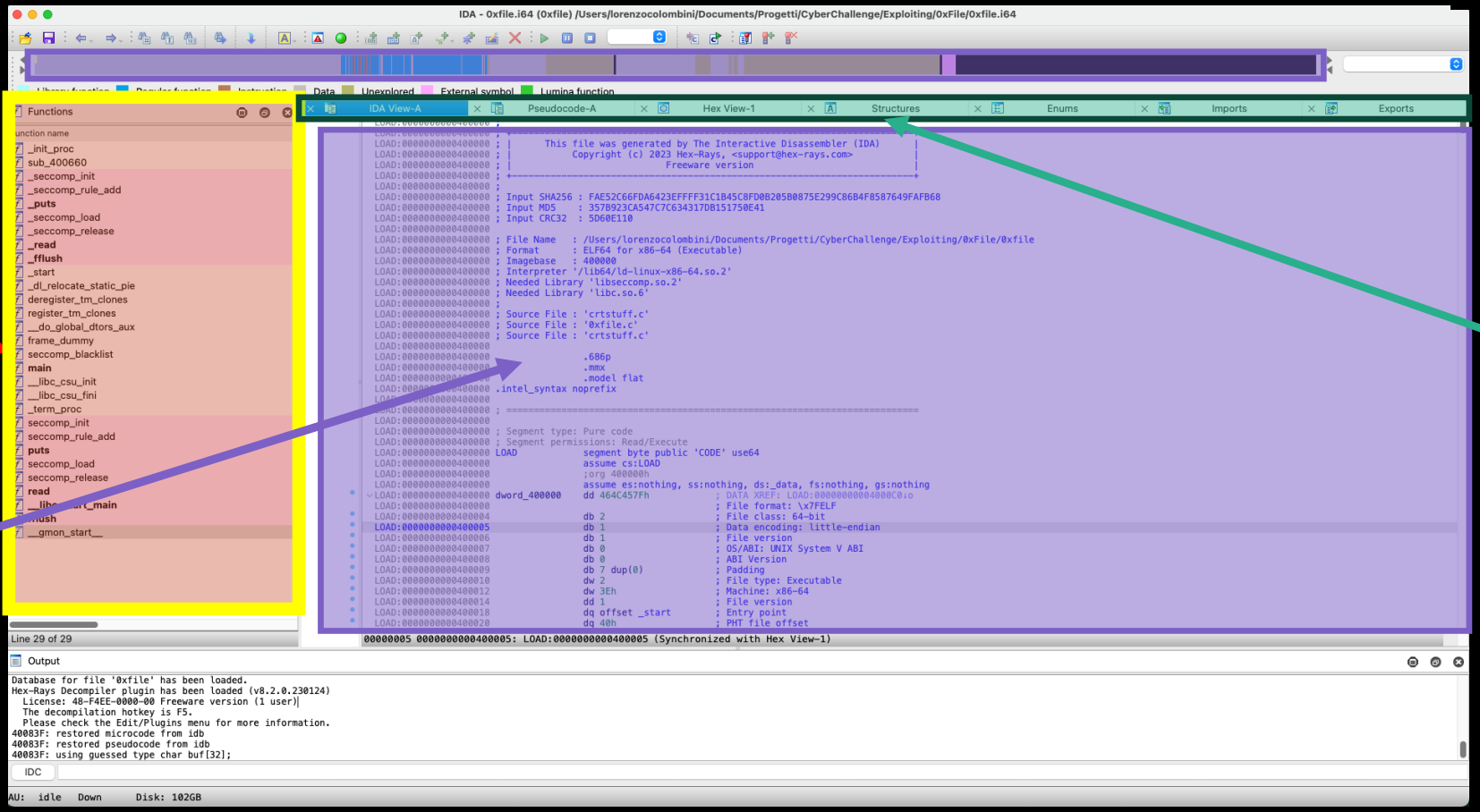


Let’s open some binary files!



Functions

Main Window:
- ASM
- Pseudocode



NavBar:
- Struct
- Hex
- Strings





TagliCorts (NEW)



Tab	Pseudocode
F5	Refresh
Shift+F12	Strings
N	Rename
Y	Set Type
H	Decimal
A	String
R	Character
X	List Cross References
Shift+F7	Segments
C	Define Code
U	Undefine Code



https://www.hex-rays.com/products/ida/support/freefiles/IDA_Pro_Shortcuts.pdf



Code outside of main

Since our first programming course, we have been told that the main function is the first and only function executed when we run a binary



Actually, the entry point of an ELF binary is _start, which calls _libc_start_main, which in turn initializes libc, calls main and finally ends the process.





Global constructors and destructors

As a standard on ELF files, there are two sections, ._init_array and ._fini_array, that contain a list of function pointers that get called before and after main.

These arrays can point to any function in the program and contain both the default libc constructor and destructor and custom code.



```
void init() __attribute__((constructor));
void fini() __attribute__((destructor));
```



In practice



By default, there is only a function both in init and fini array.

If there are other ... Someone put them there!

Ctrl+S →
→ .init_array

```
.init_array:00000000004E27D0 ; ===========================================================================
.init_array:00000000004E27D0
.init_array:00000000004E27D0 ; Segment type: Pure data
.init_array:00000000004E27D0 ; Segment permissions: Read/Write
.init_array:00000000004E27D0 _init_array     segment qword public 'DATA' use64
.init_array:00000000004E27D0                 assume cs:_init_array
.init_array:00000000004E27D0 __frame_dummy_init_array_entry dq offset frame_dummy
.init_array:00000000004E27D0                                         ; __libc_start_main_impl+95E↑o ...
.init_array:00000000004E27D0 _init_array     ends                    ; Alternative name is '__init_array_start'
.init_array:00000000004E27D0
.fini_array:00000000004E27D8 ; ===========================================================================
.fini_array:00000000004E27D8
.fini_array:00000000004E27D8 ; Segment type: Pure data
.fini_array:00000000004E27D8 ; Segment permissions: Read/Write
.fini_array:00000000004E27D8 _fini_array     segment qword public 'DATA' use64
.fini_array:00000000004E27D8                 assume cs:_fini_array
.fini_array:00000000004E27D8 __do_global_dtors_aux_fini_array_entry dq offset __do_global_dtors_aux
.fini_array:00000000004E27D8                                         ; __libc_start_main_impl+9A2↑o
.fini_array:00000000004E27D8 _fini_array     ends                    ; Alternative name is '__fini_array_start'
.fini_array:00000000004E27D8
.data.rel.ro:00000000004E27E0 ; ===========================================================================
```





Extract Data



We need to use the data we find in IDA

```
.data:0000000000004020 ; _BYTE matrix[104]
.data:0000000000004020 matrix          db 0Ah, 3 dup(0), 7Eh, 3 dup(0), 13h, 3 dup(0), 3Eh, 3 dup(0)
.data:0000000000004020                                   ; DATA XREF: check+122↑o
.data:0000000000004030                 db 50h, 3 dup(0), 70h, 3 dup(0), 5Eh, 3 dup(0), 6Fh, 3 dup(0)
.data:0000000000004040                 db 1Eh, 3 dup(0), 41h, 3 dup(0), 0CBh, 3 dup(0), 68h, 3 dup(0)
.data:0000000000004050                 db 38h, 3 dup(0), 92h, 3 dup(0), 9Dh, 3 dup(0), 26h, 3 dup(0)
.data:0000000000004060                 db 51h, 3 dup(0), 64h, 3 dup(0), 5Eh, 3 dup(0), 36h, 3 dup(0)
.data:0000000000004070                 db 9Eh, 3 dup(0), 0Ah, 3 dup(0), 33h, 3 dup(0), 9, 3 dup(0)
.data:0000000000004080                 db 7Fh, 3 dup(0), 0F7h, 3 dup(0), 0F3h, 3 dup(0), 0DFh
.data:000000000000408D                 db 3 dup(0), 0F5h, 3 dup(0), 0ADh, 3 dup(0), 0FBh, 3 dup(0)
.data:000000000000409C                 db 0EEh, 3 dup(0), 0E8h, 3 dup(0), 4Fh, 3 dup(0), 0DDh
.data:00000000000040A9                 db 3 dup(0), 78h, 3 dup(0), 96h, 3 dup(0), 22h, 3 dup(0)
.data:00000000000040B8                 db 16h, 3 dup(0), 31h, 3 dup(0), 0Fh, 3 dup(0), 0C7h, 3 dup(0)
.data:00000000000040C8                 db 8Ch, 3 dup(0), 0C9h, 3 dup(0), 0E1h, 3 dup(0), 0D3h
```

and we do not like HEX ☹





Extract Data



First, you do:
- Y → set the right type
- D → set the right size
- * → convert to array

Remember:
- H → toggle Decimal / HEX
- R → toggle Char / HEX

```
; int matrix[104]
matrix          dd 0Ah, 7Eh, 13h, 3Eh, 50h, 70h, 5Eh, 6Fh, 1Eh, 41h, 0CBh
                                        ; DATA XREF: check+122↑o
                dd 68h, 38h, 92h, 9Dh, 26h, 51h, 64h, 5Eh, 36h, 9Eh, 0Ah
                dd 33h, 9, 7Fh, 0F7h, 0F3h, 0DFh, 0F5h, 0ADh, 0FBh, 0EEh
                dd 0E8h, 4Fh, 0DDh, 78h, 96h, 22h, 16h, 31h, 0Fh, 0C7h
                dd 8Ch, 0C9h, 0E1h, 0D3h, 0AFh, 0DBh, 23h, 0FBh, 0DBh
                dd 8Eh, 34h, 0DDh, 0C5h, 0CDh, 5Eh, 0ACh, 0EBh, 29h, 8Ch
                dd 0B3h, 0D9h, 37h, 0Bh, 59h, 0B1h, 60h, 31h, 1Eh, 92h
                dd 0E0h, 86h, 3Fh, 47h, 0A0h, 6Ch, 9Ah, 4Ah, 32h, 76h
                dd 90h, 2Ch, 8, 0D2h, 8Fh, 67h, 0ADh, 0A2h, 0DBh, 75h
                dd 0ADh, 75h, 8, 0CDh, 0A2h, 0C3h, 0B5h, 0E4h, 9Dh, 4 dup(0)
```





Extract Data



Then, do:

~~*Shift+E*~~ → Export Data

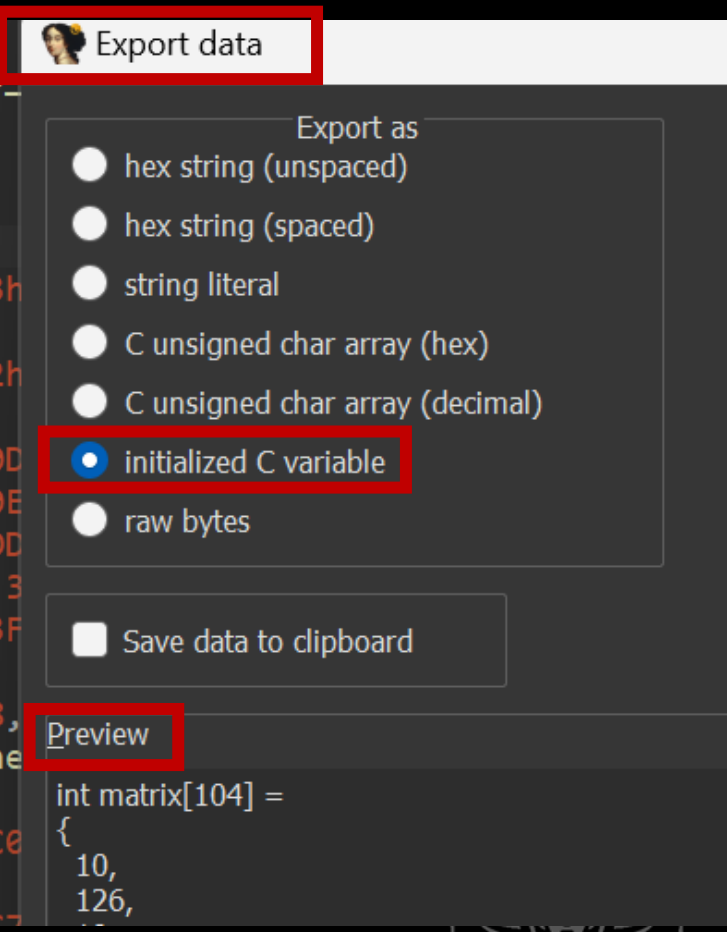
Export as ‘initialized’

Copy it from Preview or export it to file.



```
; void *_dso_handle
__dso_handle    dq offset __dso

                align 20h
                public matrix
; int matrix[104]
matrix          dd 0Ah, 7Eh, 13h
                dd 68h, 38h, 92h
                dd 33h, 9, 7Fh,
                dd 0E8h, 4Fh, 0D
                dd 8Ch, 0C9h, 0E
                dd 8Eh, 34h, 0DD
                dd 0B3h, 0D9h, 3
                dd 0E0h, 86h, 3F
                dd 90h, 2Ch, 8,
                dd 0ADh, 75h, 8,
                public checkyche
; _DWORD checkycheck[40]
checkycheck     dd 0FEF5h, 0E0C0
```





Hands on



Struct

Structures are a way to group several related variables into one place.

They can contain many different types of data and are used to model more complex data.

```
struct Brooom {
    char brand[50];
    char model[50];
    int year;
};
```



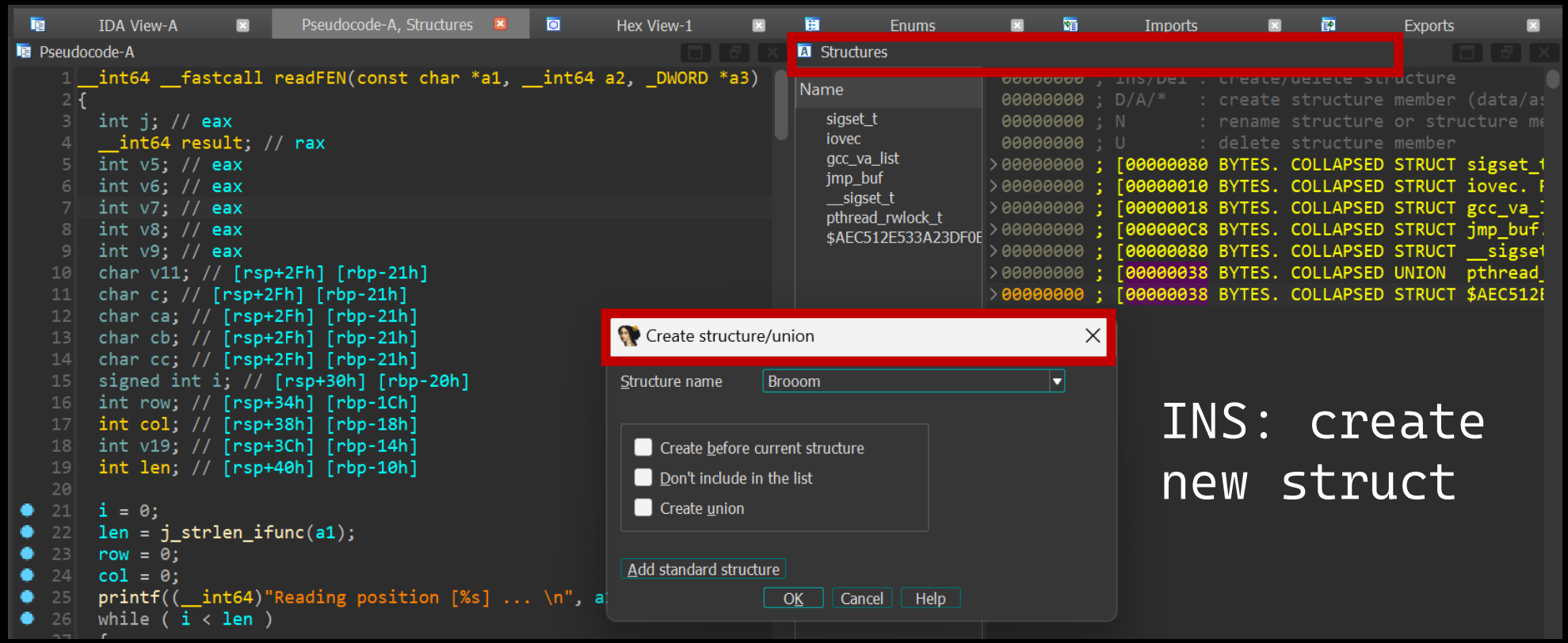




Struct - IDA



IDA does not recognize struct in pseudocode, so we need to define them and set them as type.



INS: create new struct

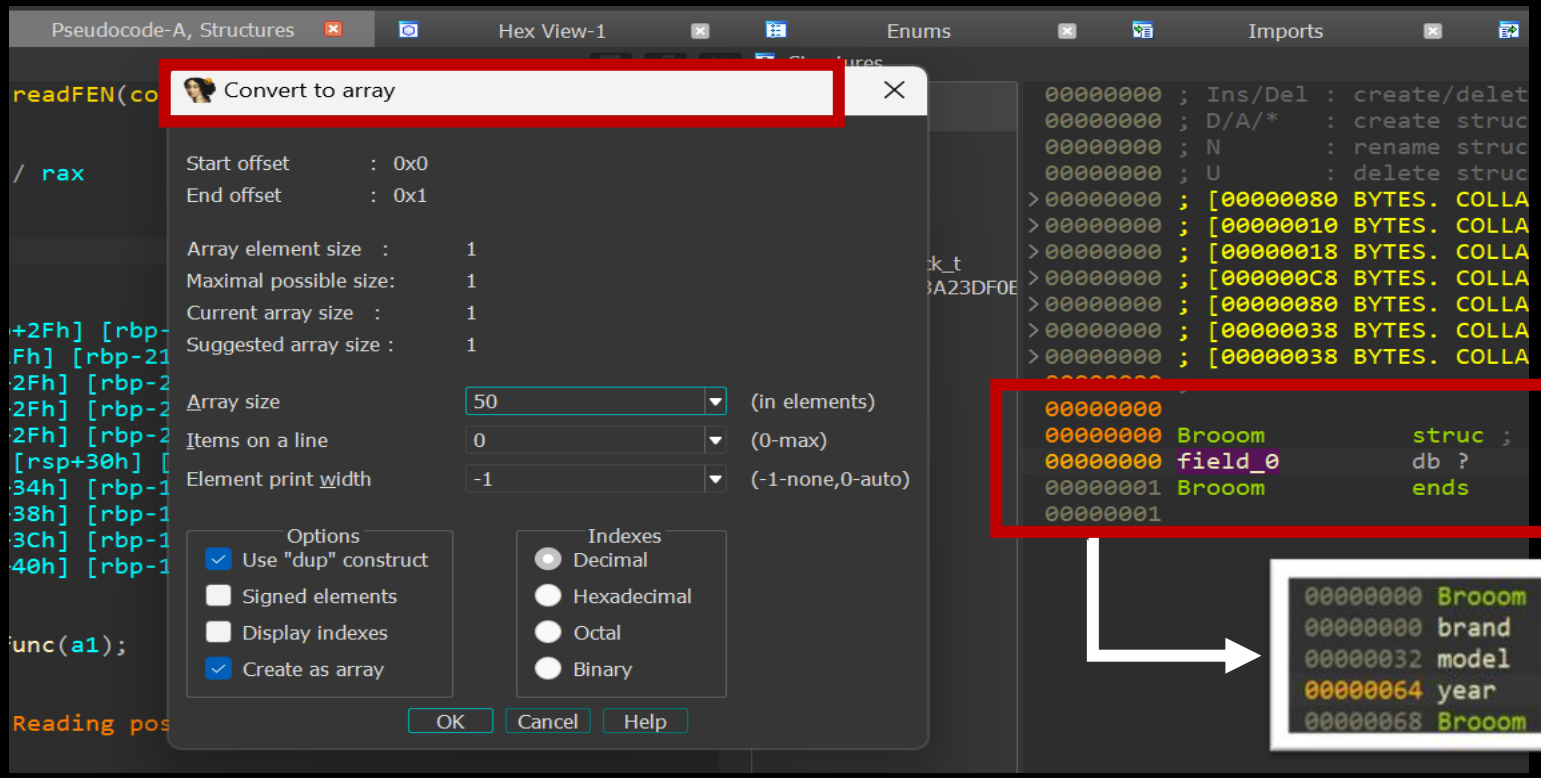




Struct - IDA



‘D’ on Struct: create new field
‘D’ on Field: change data Byte-Word-DoubleW-QuadW
‘Shift + *’ : convert to array

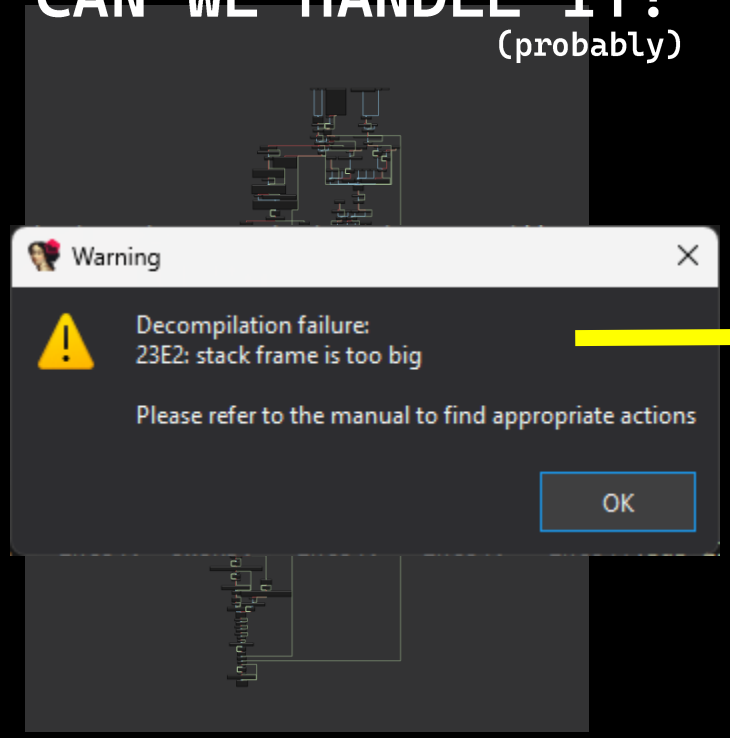


Hands on



WHAT ABOUT DYNAMIC REVERSING??

CAN WE HANDLE IT?
(probably)





WHAT IF WE COULD RUN AND DEBUG THE CODE WE ARE LOOKING AT?



STOP TALKING
GO FLAGGING

